MOVEO

An Enterprise-Scale Event-Driven
Distributed Travel Booking Platform

TEAM MEMBERS

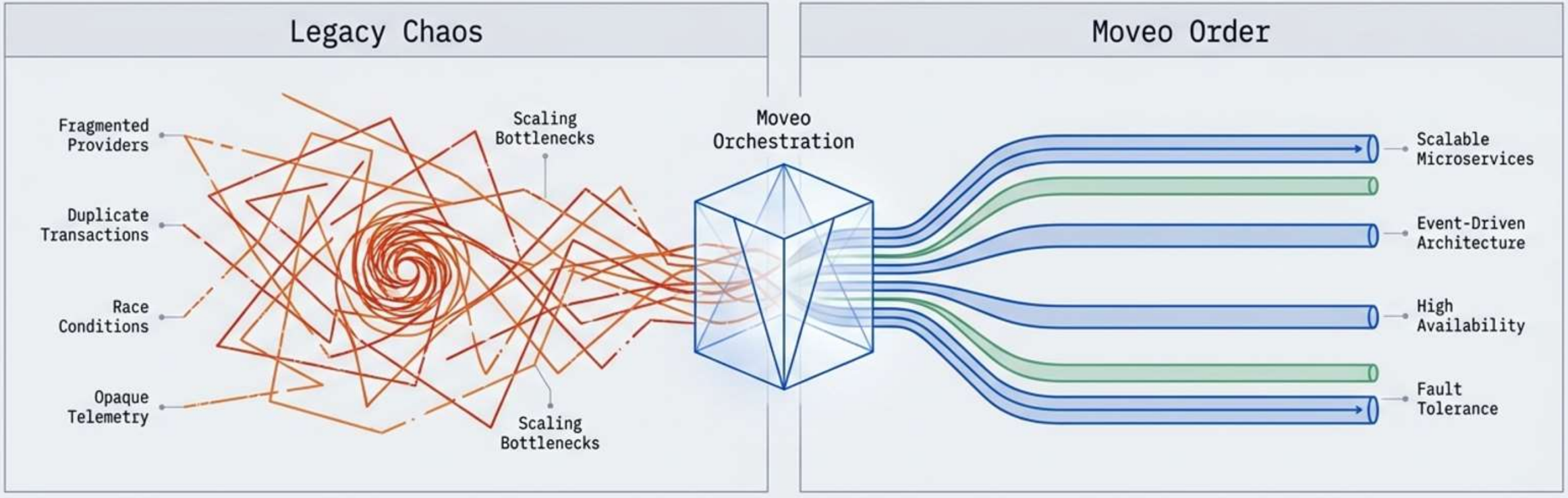
Aarush Saxena	12584200001	MCA Sec A
Aditya Gaur	12584200012	MCA Sec A
Aryan Shekhar	12584200047	MCA Sec A

UNDER THE SUPERVISION OF

Dr. Sachendra Singh Chauhan
Assistant Professor

Group No. 22 | Session 2026–27

Resolving the Inherent Chaos of Legacy Travel Booking



The Enterprise Challenges

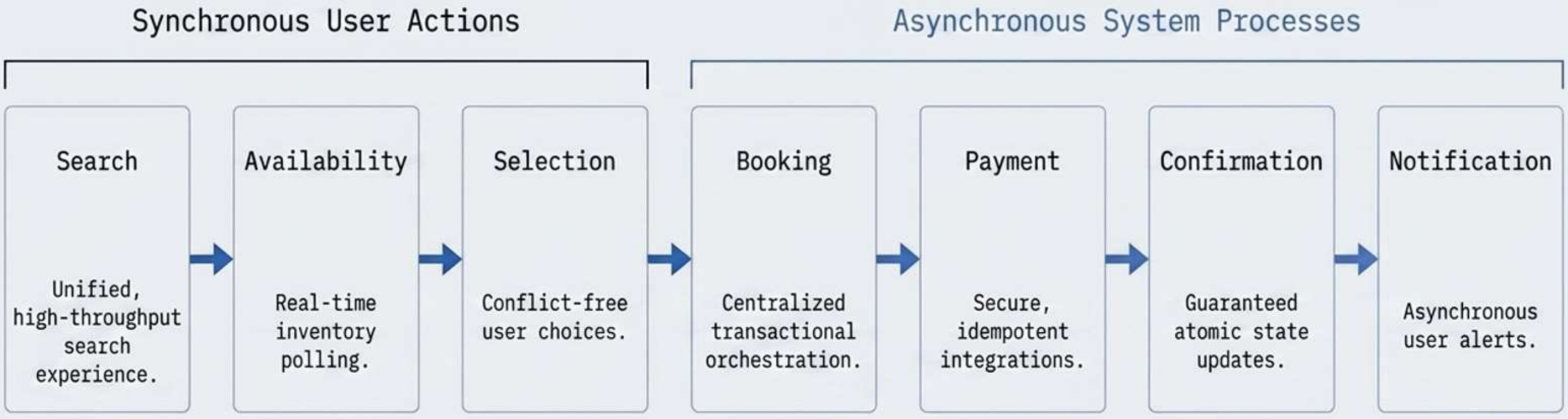
- Fragmented experiences across disparate providers.
- Inconsistent distributed booking and payment workflows.
- Duplicate transactions during high-concurrency spikes.
- Inability to independently scale search vs. booking.
- Limited distributed-system observability.

The Moveo Solution

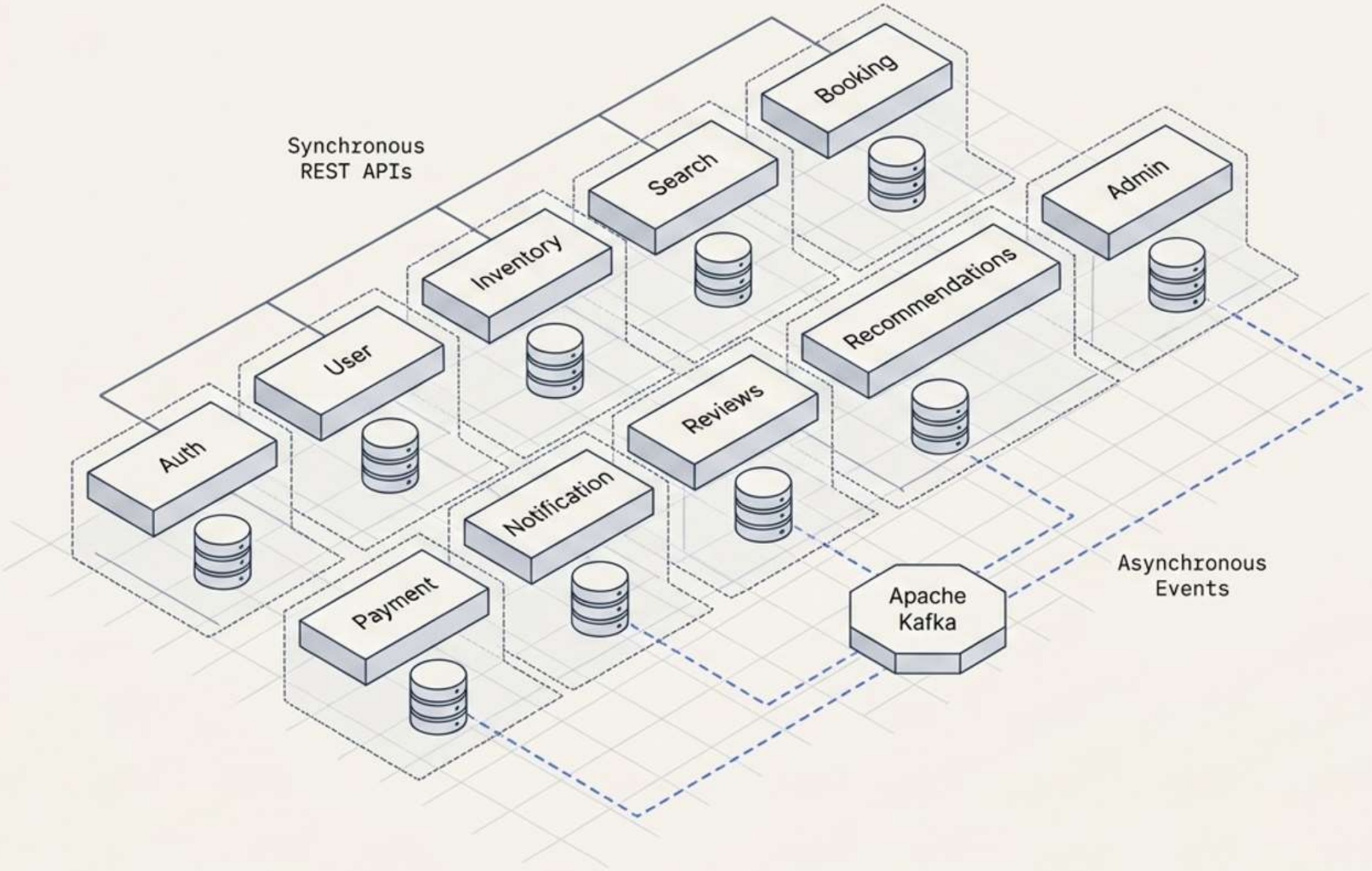
An uncompromising approach to scalable microservices, fault tolerance, and event-driven architecture to guarantee high availability and secure enterprise operations.

Orchestrating a Unified Travel Booking Engine

Unified Modalities: Flights ✈️ | Hotels 🏨 | Buses 🚌



Decentralized Control via Isolated Microservices



Architectural Imperatives

Database-per-Service: Strict physical data isolation.

Loose Coupling: Independent deployment lifecycles.

Fault Isolation: Walled-off blast radiuses preventing cascading failures.

Horizontal Scalability: Independent resource provisioning via Redis & REST/Kafka.

A High-Throughput Cloud-Native Technology Stack

Frontend (The Interface)

Next.js • React • TypeScript • Redux Toolkit • Tailwind CSS
Reactive, state-managed user interfaces.

Backend (The Engine)

Python + FastAPI:
High-performance, async-native API routing.

Messaging (The Nervous System)

Apache Kafka: Durable, decoupled event streaming.

Data (The Memory)

PostgreSQL:
Relational truth.

Redis:
In-memory distributed caching.

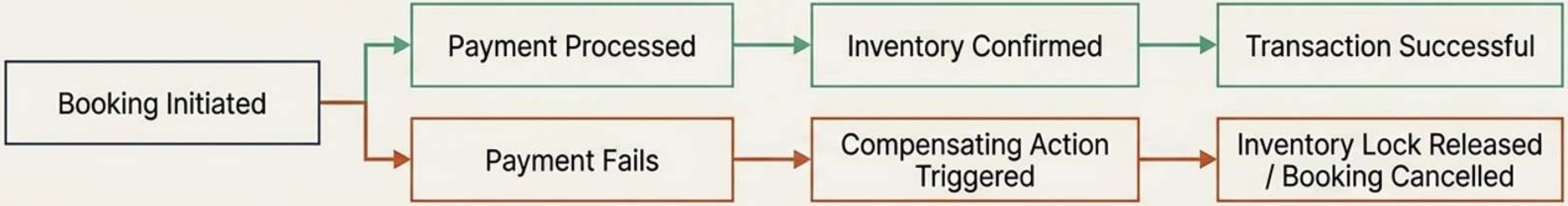
Elasticsearch:
Lightning-fast complex querying.

Engineered for Scale

Every layer of the Moveo stack is selected to eliminate bottlenecks, supporting high-throughput workloads from initial client request down to physical disk persistence.

Safeguarding Consistency in Distributed Transactions

The Saga Pattern / Rollback Flow



Risk vs. Resolution Matrix

Inherent System Risk	Moveo Architectural Pattern	Resolution Mechanism
Multi-Service Failures	`Saga Pattern`	Coordinates transactions & orchestrates compensating rollback actions.
Database/Event Inconsistency	`Transactional Outbox`	Reliably publishes state changes as immutable events.
Inventory Race Conditions	`Distributed Locks`	Protects critical, highly concurrent operations.
Duplicate Payments	`Idempotency`	Ensures safe API retries without double-charging.

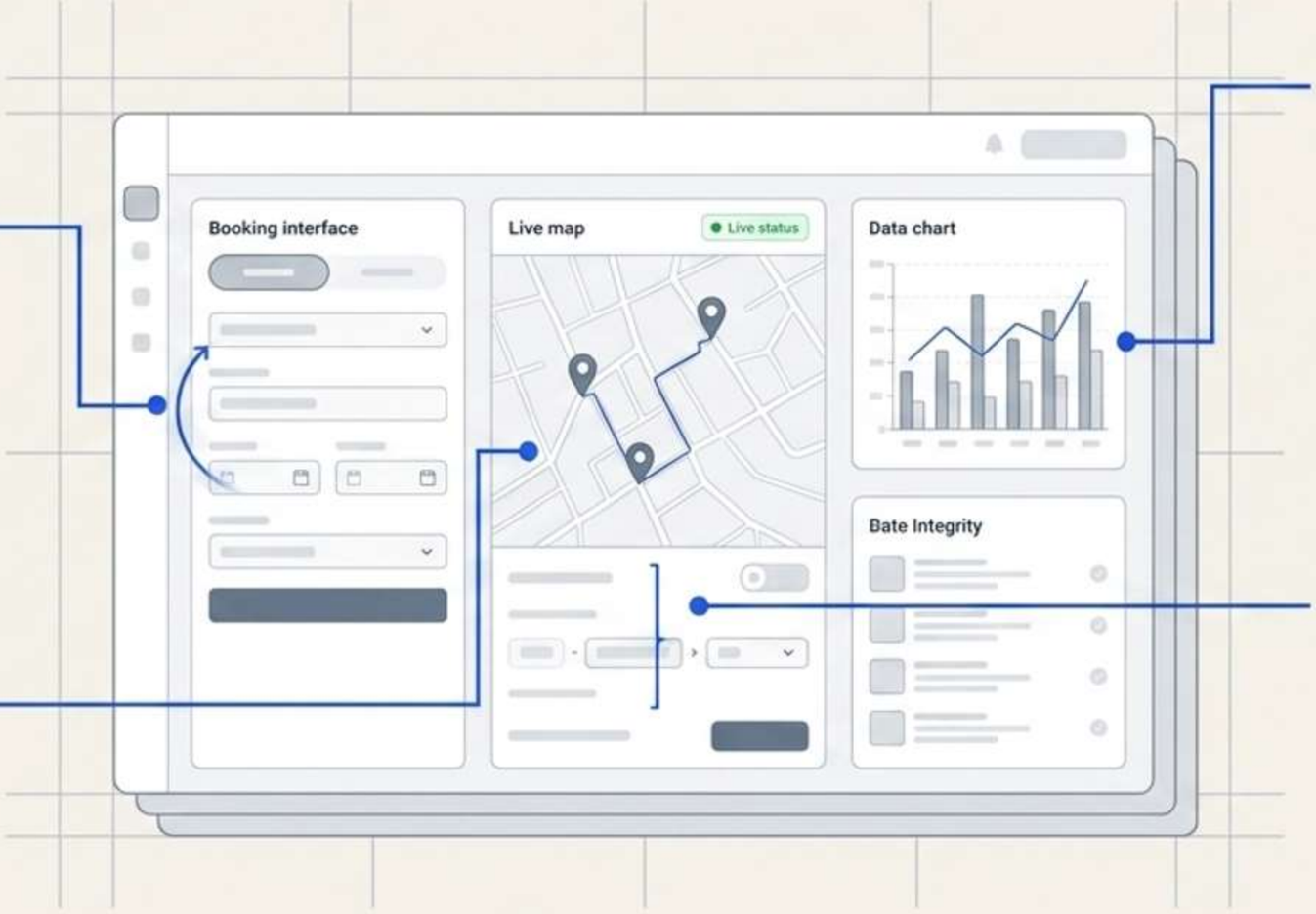
Delivering Real-Time, High-Performance User Experiences

Fluid Interactions

GSAP powers smooth, non-blocking animations for a premium feel.

Real-Time Sync

Socket.IO enables live booking availability updates without polling.



Data Visibility

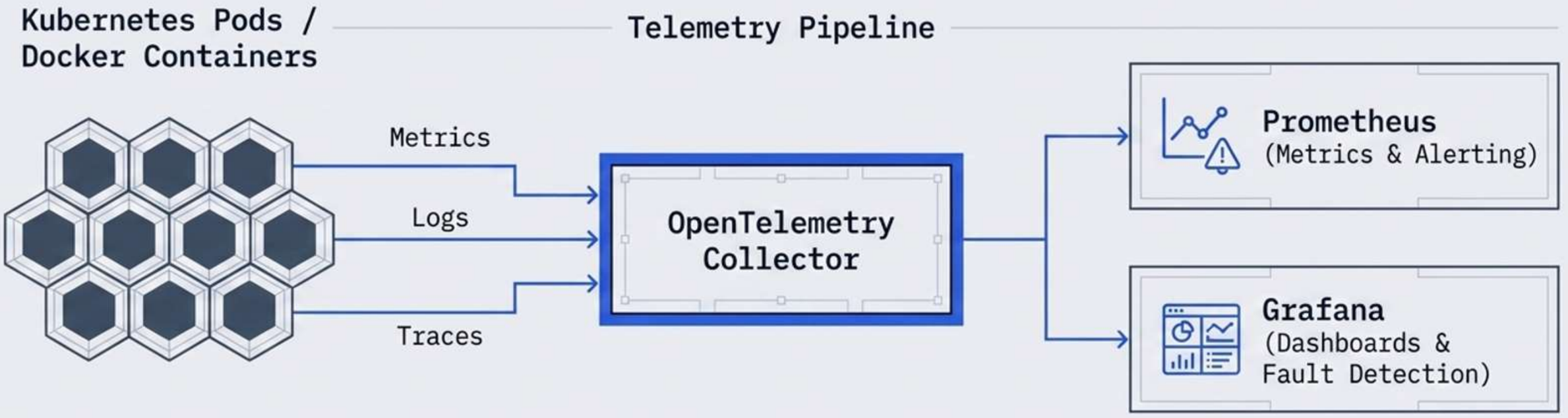
Recharts drives interactive, client-side dashboards and analytics.

State Integrity

Redux Toolkit ensures centralized, predictable state management across the complex unified search journey.

Frontend Stack Integration: Next.js | React | TypeScript | Redux Toolkit | Tailwind CSS

End-to-End Observability and Orchestration



Containerization & Orchestration

- **Docker**: Immutable, environment-agnostic artifacts.
- **Kubernetes**: Automated horizontal scaling, self-healing, and zero-downtime deployments.

The Observability Triad

- **Prometheus**: Granular service-health metrics.
- **OpenTelemetry**: Distributed request tracing across asynchronous Kafka/Saga boundaries.
- **Grafana**: Centralized performance monitoring and production readiness.

Architectural Synthesis and Core Achievements



Cloud-Native Microservices

11 distinct, decoupled operational domains.



Event-Driven Architecture

Asynchronous, Kafka-backed communication.



Reliable Distributed Transactions

Guaranteed consistency via Saga and Outbox patterns.



Database-per-Service Isolation

Complete physical separation of domain data.



High-Concurrency Protection

Distributed locks defending against race conditions.



Real-Time Frontend Experience

Socket.IO & Next.js powering live updates.



K8s Scalability & Observability

End-to-end tracing and horizontal autoscaling.

Enterprise Value and Platform Resiliency

Capability-to-Value Matrix

Capability	Value / Business Outcome
Scalability	Independently scale high-demand search or booking workloads.
Reliability	Resilient workflows with walled-off fault isolation domains.
Consistency	Zero data loss via Saga, Outbox, locking, and idempotency guarantees.
Performance	Sub-second latency via Redis caching, Kafka streaming, and Elasticsearch.
Security	Tightly controlled access boundaries across isolated services.
Observability	Immediate fault detection via comprehensive metrics and tracing.
Maintainability	Independent deployment lifecycles for modular feature shipping.

Moveo demonstrates how modern cloud-native distributed architecture can transform a complex travel booking ecosystem into a scalable, resilient, observable, and enterprise-ready platform.